THE UNLOX ACADEMY

Supervised ML Classification Project - Project Spec & Data Dictionary

QuickCart Warehouse Inventory Stockout Risk

Multi-Table Classification - Predicting Safe / At-Risk / Imminent per SKU per Store per Day

QuickCart is a quick-commerce dark-store operator (same brand used in the Week 8 K-Means customer clustering project). This project asks a different question: for every product, in every store, on every day - will we run out of stock before the next supplier delivery arrives?

This is a genuinely operational problem, not a toy dataset. It combines rolling sales velocity, supplier lead-time uncertainty, festival demand spikes, and per-store buffer differences - the same kind of signal mix a real inventory-planning team works with daily.

NOTE

This dataset is custom-built, unpublished, and paired with locked sanity numbers below, so results can be verified but not looked up.

It also matches our exact pedagogical shape: multi-table joins, a natural 3-class imbalance, and real feature-engineering payoff (rolling averages, category flags, supplier reliability) - none of which off-the-shelf datasets are built to teach.

1. The Business Problem

QuickCart runs 12 dark stores across 6 Indian metros, stocking 60 SKUs across 8 categories, sourced from 15 suppliers. Every day, for every SKU at every store, the inventory team needs to know: is this item Safe, At-Risk, or Imminent (about to run out before the next delivery arrives)?

Get this wrong in one direction and shelves sit empty - lost sales, annoyed customers, and (per the Week 9 NLP project) exactly the kind of negative review that drags down MasalaMart-style sentiment scores. Get it wrong in the other direction and the store over-orders, tying up capital and warehouse space on stock that sits too long.

The target variable

Class	Definition	Share of data
Safe	Days of stock cover exceeds the wait for replenishment by 3+ days	65.4%
At-Risk	Days of cover is within 3 days of the replenishment wait - tight but not critical	24.0%
Imminent	Stock will run out (or already has) before the next delivery arrives	10.6%

This mirrors real operational triage systems: a rare, high-cost ‘Imminent’ class sitting inside a much larger ‘everything is fine’ population - the same imbalance shape students saw with ER-style triage or fraud detection, but framed around a domain nobody’s over-taught yet.

2. Schema - 5 Tables

Deliberately kept to 5 tables - enough relational complexity to force real joins, not so much that data assembly becomes its own multi-week detour.

dim_stores.csv (12 rows)

Column	Type	Notes
store_id	string	PK. ST01–ST12
city	string	6 metros, 2 stores each - clean join key
city_display	string	Same as city, EXCEPT 2 rows have inconsistent casing (planted - see $4)
city_tier	string	Tier1 for all 12 (kept uniform - not a modeling feature)
store_size	string	Small / Medium / Large
sqft	int	Store floor area
opened_year	int	2022–2025
baseline_daily_orders	int	Used to derive demand share across stores

dim_skus.csv (60 rows)

Column	Type	Notes
sku_id	string	PK. SKU001–SKU060
sku_name	string	Product name
category	string	8 categories (F&V, Dairy, Snacks, Beverages, Personal Care, Home Care, Staples, Bakery)
unit_price_inr	float	₹15–₹450 depending on category
shelf_life_days	int	1–730 depending on category
is_perishable	Y/N	Feature: perishables run tighter cover targets
festive_relevant	Y/N	Feature: Snacks, Beverages, Bakery see festival demand spikes
popularity_tier	string	High / Medium / Low - drives baseline demand
supplier_id	string	FK → dim_suppliers

dim_suppliers.csv (15 rows)

Column	Type	Notes
supplier_id	string	PK. SUP01–SUP15
supplier_name	string	Fictional distributor names
categories_supplied	string	1–2 categories per supplier
reliability_score	float 0–1 (or text 'N/A')	3 of 15 stored as literal text 'N/A' - see $4 gotcha
base_lead_time_days	int	1–5 days
lead_time_variance_days	float	Feeds the reorder delay simulation

dim_events.csv (30 rows - one per day)

Column	Type	Notes
date	date	2026-10-01 to 2026-10-30 (designed teaching calendar)
event_name	string	'Regular Day', 'Diwali Week', 'Weekend Flash Sale'
event_type	string	none / festival / promo
demand_multiplier_festive	float	Applied to festive_relevant SKUs only
demand_multiplier_other	float	Smaller bump applied to all other SKUs

fact_inventory_daily.csv (21,600 rows - the modeling table)

One row per (store, SKU, date) - 12 stores × 60 SKUs × 30 days = 21,600 rows.

Column	Type	Notes
date, store_id, sku_id, supplier_id	—	Composite grain + FKs
opening_stock, closing_stock	float	Units on hand at start/end of day
units_demanded, units_sold	int/float	Demanded may exceed sold when stock runs out
reorder_point	float	Threshold that triggers a new order
reorder_placed	Y/N	Whether an order was placed that day
lead_time_days_expected	int	Supplier's base lead time
lead_time_days_actual	float, mostly missing	Only populated on reorder days (92.1% missing by construction - see $5)
sales_velocity_7d	float	Rolling 7-day average units sold

Column	Type	Notes
days_of_cover	float	closing_stock / sales_velocity_7d
is_festival_week	Y/N	Flag for the Diwali-week demand spike
stockout_risk	string	TARGET — Safe / At-Risk / Imminent

3. Locked Sanity Numbers

Verify these after loading, before doing any modeling. If your numbers don't match, something broke in the join or the load.

```
Row counts:
  dim_stores.csv:            12 rows
  dim_skus.csv:               60 rows
  dim_suppliers.csv:          15 rows
  dim_events.csv:             30 rows
  fact_inventory_daily.csv:   21,600 rows  (12 x 60 x 30)

Target distribution (fact_inventory_daily.csv):
  Safe:      65.42%  (14,131 rows)
  At-Risk:   24.01%  (5,186 rows)
  Imminent:  10.57%  (2,283 rows)

Date range: 2026-10-01 to 2026-10-30 (30 unique dates)
Diwali Week: Oct 22-26, 2026 (demand multiplier peaks at 2.6x on Oct 25)
Weekend Flash Sale: Oct 11, 2026 (1.3x demand, all categories)
```

```
Money-moment signals (verify these before building your report):

Festival week effect:
  Imminent rate, non-festival days:  9.51%
  Imminent rate, festival week:      23.31%   (2.45x spike)

Supplier reliability effect:
  Imminent rate, low reliability  (<0.75):     15.83%
  Imminent rate, mid reliability (0.75-0.85):   6.30%
  Imminent rate, high reliability (>=0.85):     3.82%

Perishable effect (smaller but real):
  Imminent rate, non-perishable SKUs:  9.3%
  Imminent rate, perishable SKUs:     12.8%

Missing lead_time_days_actual: 19,899 of 21,600 rows (92.1%)
  — by construction, only populated on days a reorder was placed
```

4. Planted Data-Quality Issues

Two deliberate quirks, both realistic, both worth a teaching moment before modeling:

1. Inconsistent city casing (dim_stores.csv)

2 of 12 stores have a city_display value that doesn't match city exactly (one upper-cased, one lower-cased). The join key (city) is clean - city_display is a display-only field that would break a naive groupby('city_display') until standardized with .str.title() or similar.

2. The 'N/A' reliability score (dim_suppliers.csv)

GOTCHA

This one is a genuine pandas gotcha, not just a planted typo.

3 of 15 suppliers have reliability_score stored as the literal text 'N/A' in the CSV. pandas' default CSV reader (pd.read_csv) treats 'N/A' as one of its built-in missing-value markers and SILENTLY converts it to NaN on load - you will not see the string 'N/A' in the DataFrame at all, even though it's sitting right there in the raw file.

This is worth demonstrating live: open the CSV in a text editor to show 'N/A' is really there, then pd.read_csv() it and show .isna() catches it anyway - but only because pandas happened to guess right. If a dataset used a DIFFERENT placeholder ('missing', '--', 'unknown'), pandas would NOT catch it automatically, and it would silently become a string category instead of a proper NaN. This is a real production bug pattern.

The fix pattern to teach: pd.read_csv(path, na_values=['N/A', 'missing', '--', 'NA', 'null'], keep_default_na=True) to be explicit rather than relying on defaults.

5. Feature Engineering Guidance

The raw fact table is intentionally NOT model-ready - that's the point. Here's the intended feature engineering path:

FEATURE ENGINEERING TIP

Suggested derived features.

reorder_gap = reorder_point - closing_stock (how far past/before the trigger point)

days_of_cover_ratio = days_of_cover / lead_time_days_expected (normalized cover)

supplier_reliability_clean = numeric reliability, 'N/A' imputed with category median

is_recent_reorder = reorder_placed within the last N days (requires a groupby-shift on store+sku)

category one-hot or target encoding - 8 categories, meaningful stockout-rate variation across them

day_of_month, days_since_festival_start - temporal features around the Diwali spike

Suggested train/test split

Because this is a daily panel (same store-SKU pairs repeated across 30 days), a random row-level split will leak information - the model can 'see' a store-SKU pair's pattern from days it was trained on and just interpolate for test days from the SAME pair. Recommend either:

- Time-based split: train on Oct 1–23, test on Oct 24–30 (includes the back half of Diwali week - a genuinely hard generalization test), or
- Group-based split: hold out entire store-SKU pairs (e.g., stratified by category) so the model is tested on combinations it has never seen

Suggested models

- Baseline: majority-class classifier (65.4% - same 'always report a baseline' lesson as W9 S3)
- Multinomial Logistic Regression - interpretable coefficients per feature per class
- Random Forest / Gradient Boosting - handles the non-linear reorder-point interactions well, gives feature_importances_
- Metric focus: recall on Imminent matters most - missing a real stockout is far costlier than a false alarm (same asymmetric-cost lesson as W9 S3's confusion matrix)

6. File Manifest

File	Rows	Purpose
dim_stores.csv	12	Store dimension
dim_skus.csv	60	Product dimension
dim_suppliers.csv	15	Supplier dimension
dim_events.csv	30	Festival/promo calendar
fact_inventory_daily.csv	21,600	Modeling table - join the above 4 tables to this one

Continuity note: QuickCart is the same brand used in Week 8 Session 1 (K-Means customer clustering). This project is a separate, unrelated dataset - no shared keys with the customer data - but keeps the quick-commerce thread going across the course.